

Time Series Forecasting with XGBoost (also with Baguettes)

Following the “Baguette Series”.

Hernández Pérez Erick Fernando

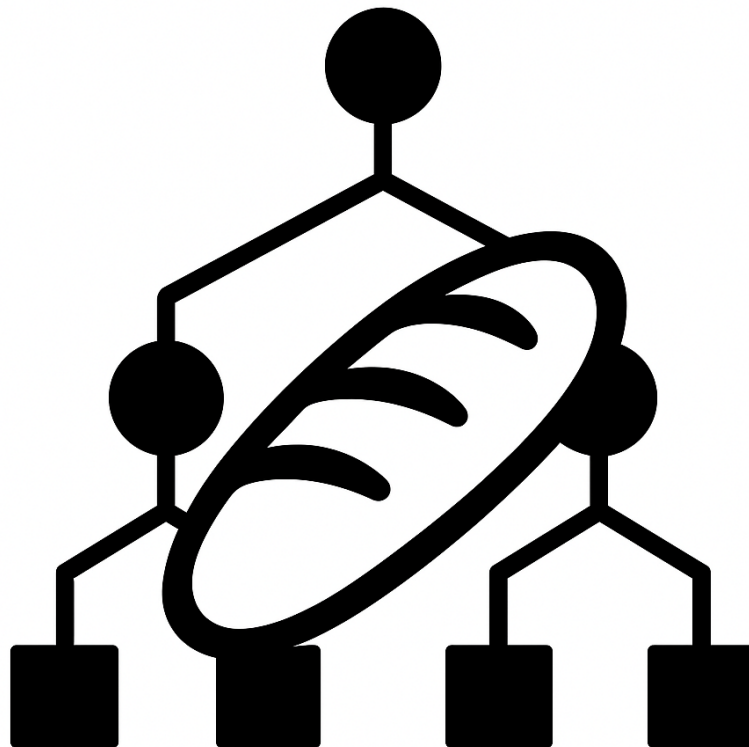


Figure 1

While it is true that time series forecasting relies heavily on solid mathematical and statistical foundations — which I highly recommend studying in depth to uncover the subtle reasons behind why things are done the way they are (and to truly understand where models like ARIMA come from) — we cannot overlook the fact that this series of articles is ultimately focused on the Artificial Intelligence perspective.

In the previous articles, we explored the key theoretical and practical concepts that pave the way for working with time series data. That foundation was essential to build a clear intuition before diving into machine learning applications.

Now, it's time to move from ‘traditional’ forecasting models to more modern, data-driven approaches. In this article, we will explore how to leverage **XGBoost** for time series forecasting — one of the most powerful and versatile algorithms in the machine learning toolbox.

To ensure continuity, we will revisit the *Baguettes* dataset used in the previous post, *The Four Basic Forecasting Models in Python You Should Know (With Baguettes)*, and demonstrate how to extend those methods into the realm of gradient-boosted trees.

What is XGBoost and Why Use It for Time Series Forecasting?

The algorithm known as **XGBoost** (eXtreme Gradient Boosting) is a high-performance implementation of the gradient-boosted decision tree (GBDT) framework. At its core it builds an ensemble of decision trees, where each new tree aims to correct the errors (residuals) of the existing ensemble via a gradient descent (and second-order) optimization of a loss function.

Key particularities of XGBoost that make it stand out:

- Regularisation built-in: XGBoost introduces explicit penalties for complexity (tree depth, leaf weights, etc), which helps control overfitting.
- Optimised training and scalability: It uses sparsity-aware algorithms, efficient cache and memory-aware blocks, and supports parallel/distributed training (even GPU) for very large datasets.
- Versatility: Though originally common for classification or regression on tabular data, XGBoost is easily applied to many supervised tasks, with fine-tunable hyperparameters and good interpretability (via feature importance, though less than a single decision tree).

While traditional time series models (e.g., ARIMA, exponential smoothing) encode domain-specific structure (lags, seasonality, stationarity), they may struggle when the real-world data exhibit complex non-linearities, interactions or external covariates. The appeal of XGBoost in a time-series forecasting context lies in several aspects:

- Through feature engineering (lagged values, rolling statistics, calendar effects, external regressors) a time series problem can be cast as a supervised regression task — making it amenable to tree-based methods like XGBoost.
- The boosting mechanism allows capturing non-linear relationships and interactions between features (for example, how lag 3 interacts with day-of-week or holiday indicators).
- The regularisation and efficient training help when you have many candidate features or when you want to avoid over-fitting to noise.
- Empirical evidence (for example, the referenced study) suggests that well-tuned tree-based models can match or outperform many more complex deep-learning architectures when properly structured.

In the paper “Do We Really Need Deep Learning Models for Time Series Forecasting?” (Elsayed, Thyseens et al., 2021) the authors compare a gradient-boosting regression tree (GBRT) baseline (essentially in the spirit of XGBoost) against a variety of deep learning models on nine time-series datasets. They show that when the time-series forecasting task is recast as a window-based regression problem and features are carefully engineered, the tree-based model actually outperforms the deep models.

This result underlines the important message: for many forecasting tasks, a well-designed feature-engineered machine-learning model (such as XGBoost) can be not only simpler to implement and interpret, but also competitive (or better) in performance compared to more “black-box” deep-learning alternatives.

Thus, in the context of this article series — where we first laid the theoretical and practical foundations of time series, and now shift toward AI-driven methods — employing XGBoost is a logical next step: it bridges familiar supervised-learning methods with the challenges of forecasting, enabling us to go beyond classical models into the machine-learning domain while retaining interpretability and efficiency.

Let’s Move to the Hands-On Part

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

df_bakery = pd.read_csv('daily_sales_french_bakery.csv', parse_dates=['ds'])
df_bakery = df_bakery.groupby('unique_id').filter(lambda x: len(x) >= 28)
df_bakery = df_bakery.drop('unit_price', axis=1)
```

```

df_baguette = df_bakery.loc[df_bakery['unique_id'] ==
    ↳ 'BAGUETTE'].set_index('ds').drop('unique_id',
    ↳ axis=1).resample('D').interpolate(method='linear')

fig, ax = plt.subplots(figsize=(15,8))

sns.lineplot(
    data=df_baguette,
    x=df_baguette.index,
    y='y',
    ax=ax,
    color='brown',
    linewidth=2
)

ax.set_title('Historical Daily Baguette Sales ', fontsize=20, weight='bold')
ax.set_xlabel('Date', fontsize=14)
ax.set_ylabel('Sales', fontsize=14)
ax.grid(True, linestyle='--', alpha=0.5)

plt.xticks(rotation=45)
ax.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:,.0f}'))

plt.tight_layout()
plt.show()

```

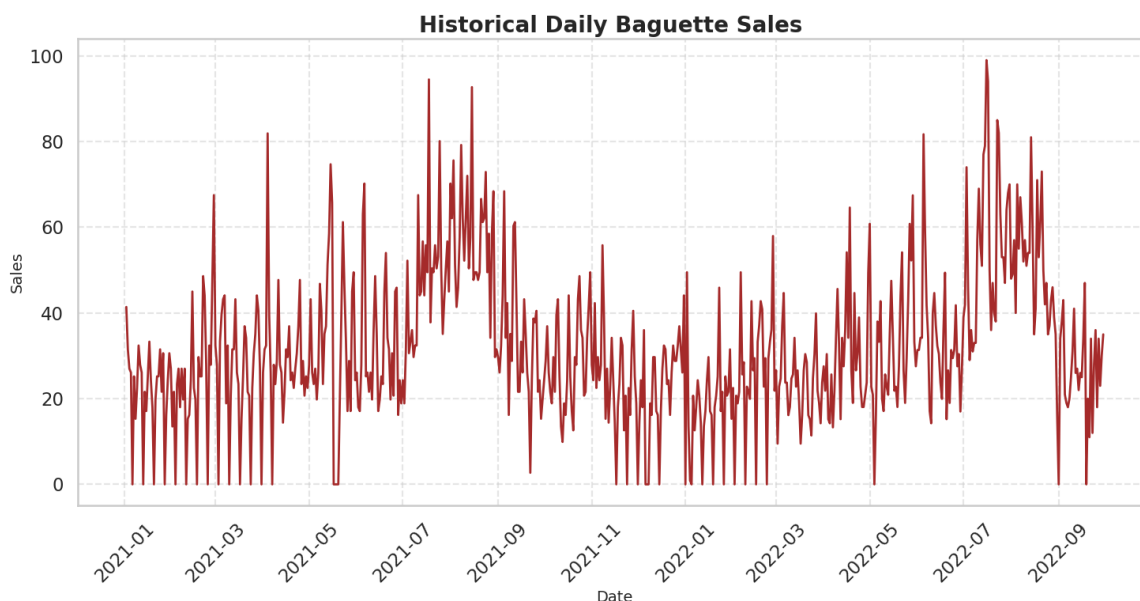


Figure 2

Feature Engineering for Time Series: Giving XGBoost the Right Perspective

Before training XGBoost, we need to reshape our time series into a form that a supervised learning model can understand. Unlike classical models such as ARIMA, tree-based algorithms do not inherently know the temporal order of observations. Therefore, we must *manually encode* the temporal structure of the data through feature engineering.

The following transformations were applied to our `df_baguette` dataset:

```

df_baguette['day'] = df_baguette.index.day
df_baguette['dayofweek'] = df_baguette.index.dayofweek
df_baguette['month'] = df_baguette.index.month
df_baguette['dayofyear'] = df_baguette.index.dayofyear
df_baguette['year'] = df_baguette.index.year

for lag in range(1, 8):
    df_baguette[f'lag_{lag}'] = df_baguette['y'].shift(lag)

df_baguette['rolling_mean_7'] = df_baguette['y'].rolling(7, min_periods=1).mean()
df_baguette['rolling_std_7'] = df_baguette['y'].rolling(7, min_periods=1).std()
df_baguette['target'] = df_baguette['y'].shift(-1)
df_baguette = df_baguette.dropna()

X = df_baguette.drop(columns=['y', 'target'])
y = df_baguette['target']

```

Let's break down what this transformation accomplishes:

- **Calendar features** (`day`, `dayofweek`, `month`, `dayofyear`, `year`) encode temporal and seasonal information. They allow the model to capture patterns such as weekly demand cycles or monthly trends — elements that are often crucial in time-dependent behavior.
- **Lag features** (`lag_1` to `lag_7`) represent the past seven days of the target variable. These features explicitly tell XGBoost what recent values looked like, enabling it to learn short-term dependencies similar to the autoregressive component in ARIMA models.
- **Rolling statistics** (`rolling_mean_7`, `rolling_std_7`) summarize recent trends and volatility. Such aggregated information helps the model identify local level shifts, bursts in variability, or stabilization periods in the data.
- **Target shifting.** By shifting the variable `y` one step ahead to create a new `target`, we transform the forecasting task into a one-step-ahead regression problem, where the features (`X`) describe the past and the label (`y`) represents the next observation to predict.

After cleaning missing values, we separate the dataset into predictors (`X`) and the forecasting target (`y`). This process effectively turns our time series into a supervised learning dataset — one where XGBoost can apply its strength in capturing non-linear interactions, seasonal dependencies, and feature relationships without any explicit assumption of stationarity.

Model Training and Hyperparameter Optimization

With our features prepared, the next step is to train and fine-tune the XGBoost model. Unlike classical time series models, XGBoost requires explicit hyperparameter tuning to balance bias, variance, and computational efficiency. In this section, we use `GridSearchCV` in combination with `TimeSeriesSplit` to find the best configuration for our data.

```

from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from xgboost import XGBRegressor

xgb = XGBRegressor(objective='reg:squarederror')

param_grid = {
    'n_estimators': [100, 200, 300, 400, 500, 1000],
    'max_depth': [3, 4, 5, 6, 7],
    'learning_rate': [0.01, 0.05, 0.1],
    'subsample': [0.6, 0.7, 0.8, 1.0]
}

```

```
tscv = TimeSeriesSplit(n_splits=5)

gsearch = GridSearchCV(
    estimator=xgb,
    param_grid=param_grid,
    scoring='neg_mean_absolute_error',
    cv=tscv,
    refit=False,
    verbose=1,
    n_jobs=-1
)

gsearch.fit(X, y)

print("Best params:", gsearch.best_params_)
print("Best MAE:", -gsearch.best_score_)
```

Fitting 5 folds for each of 360 candidates, totalling 1800 fits

Best params: {'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 300, 'subsample': 0.6}

Best MAE: 10.58068087394421

Let's break down each component of this process:

- **XGBRegressor:** This is the regression implementation of XGBoost, configured here with the objective function `'reg:squarederror'`, which minimizes the mean squared error between predicted and actual values.
- **Hyperparameter grid:** The dictionary `param_grid` defines the search space for tuning the model's complexity and learning behavior:
 - `n_estimators` controls the number of boosting rounds (trees). More trees can capture more complex patterns but may lead to overfitting.
 - `max_depth` limits the maximum depth of each tree, managing the model's complexity.
 - `learning_rate` shrinks the contribution of each tree, providing a trade-off between training speed and generalization.
 - `subsample` determines the fraction of samples used to grow each tree, introducing randomness to reduce overfitting.
- **TimeSeriesSplit:** Unlike random cross-validation, `TimeSeriesSplit` preserves temporal order. Each fold trains on past data and validates on future observations, mimicking real-world forecasting conditions and avoiding data leakage.
- **GridSearchCV:** This tool exhaustively tests all hyperparameter combinations defined in `param_grid` using the specified cross-validation strategy. We use `neg_mean_absolute_error` as the scoring metric because MAE is robust to outliers and interpretable in forecasting contexts.
- **Model selection:** After training, the grid search reports the best parameter combination (`best_params_`) and its corresponding validation error (`best_score_`). The lower the MAE, the better the model generalizes to unseen temporal segments.

This process ensures that our final XGBoost configuration is well-tuned for the temporal structure of the data, providing a strong foundation before moving into the evaluation and interpretation phase. It also highlights an important principle in machine learning for time series: *model performance depends not only on the algorithm itself, but on how well its parameters and validation strategy respect the time-dependent nature of the data.*

Final Model Training and Performance Visualization

After identifying the best hyperparameters through cross-validation, we can now train a final XGBoost model on the training portion of our dataset and evaluate its performance on unseen data.

This step simulates a realistic forecasting scenario where the model is fit on historical data and then tested on future observations.

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from xgboost import XGBRegressor

# -----
# Train/test split
# -----
train_X = X.loc[X.index < '2022-03-01'].copy()
train_y = y.loc[y.index < '2022-03-01'].copy()

test_X = X.loc[X.index >= '2022-03-01'].copy()
test_y = y.loc[y.index >= '2022-03-01'].copy()

# -----
# Train model
# -----
model = XGBRegressor(
    learning_rate=0.01,
    max_depth=3,
    n_estimators=300,
    subsample=0.6,
    random_state=42
)

model.fit(train_X, train_y)

# -----
# Ensure same feature order
# -----
model_features = model.get_booster().feature_names
test_X = test_X[model_features].copy()

# -----
# Predict
# -----
test_pred = model.predict(test_X)

# Crear DataFrame con resultados
results = pd.DataFrame({
    'y_true': test_y,
    'y_pred': test_pred
}, index=test_y.index)

# -----
# Plot real vs predicted
# -----
fig, ax = plt.subplots(figsize=(15, 8))

sns.lineplot(
    data=results,
    x=results.index,
    y='y_true',
    ax=ax,
    color='brown',
```

```

        linewidth=2,
        label='Actual'
    )

    sns.lineplot(
        data=results,
        x=results.index,
        y='y_pred',
        ax=ax,
        color='green',
        linewidth=2,
        label='Predicted'
    )

    ax.set_title('Daily Baguette Sales Forecast (Test Period)', fontsize=20, weight='bold')
    ax.set_xlabel('Date', fontsize=14)
    ax.set_ylabel('Sales', fontsize=14)
    ax.grid(True, linestyle='--', alpha=0.5)

    plt.xticks(rotation=45)
    ax.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:,.0f}'))

    ax.legend()
    plt.tight_layout()
    plt.show()

```

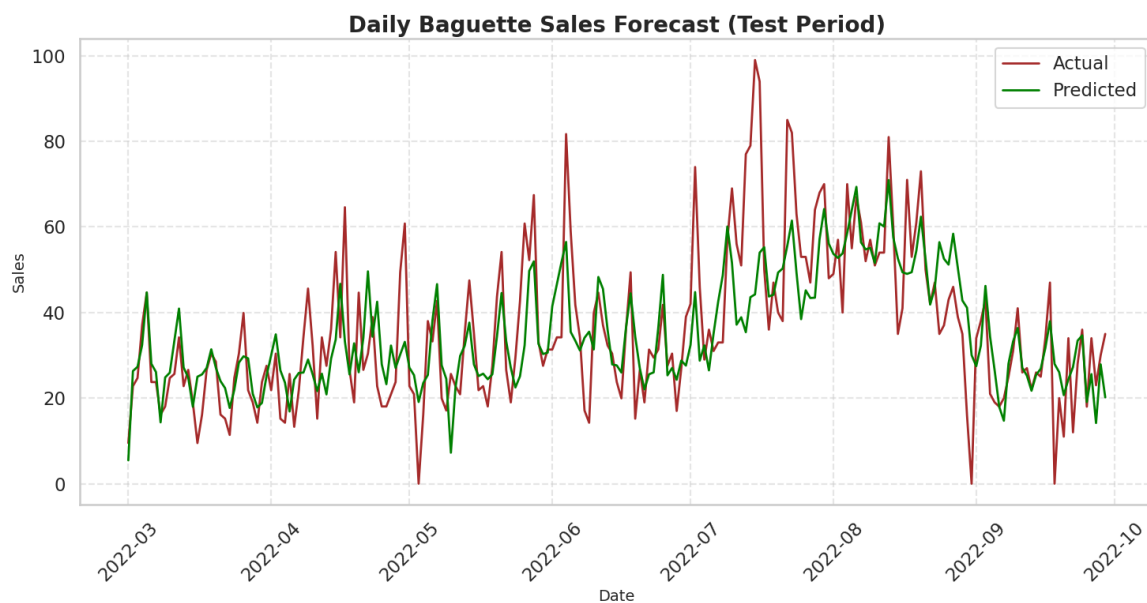


Figure 3

We manually split the dataset into a training and test period, ensuring that the temporal order of observations is respected — a crucial step for any time series experiment. This mirrors the way forecasts are generated in real operational settings: the model learns from the past and predicts what comes next.

- The chosen parameters (`learning_rate = 0.01`, `max_depth = 3`, `n_estimators = 300`, `subsample = 0.6`) were selected from the grid search, balancing model complexity and generalization.
- The prediction step produces a time-indexed DataFrame containing both actual (`y_true`) and predicted (`y_pred`) values.

- The visualization uses **seaborn** to display how closely the model tracks the actual series during the test window.

The resulting chart illustrates the model's ability to capture temporal dynamics (including trends, fluctuations, and seasonality) while maintaining smooth, data-driven forecasts.

Although XGBoost does not inherently model time dependence like ARIMA or LSTM, well-designed features and tuned hyperparameters allow it to perform competitively in practical forecasting tasks.

This hands-on stage marks the culmination of our journey: transforming a raw time series into a predictive, machine-learning-powered system that can generalize beyond historical data. In the next steps, we could further extend this analysis with error diagnostics, feature importance visualization, or even hybrid ensemble approaches combining statistical and ML methods.

References

- [1] Auffarth, B. (2021). *Machine Learning for Time-Series with Python*. Packt.
- [2] Elsayed, S., Thyssens, D., Rashed, A., Jomaa, H. S., & Schmidt-Thieme, L. (2021). *Do we really need deep learning models for time series forecasting?* arXiv. <https://arxiv.org/abs/2101.02118>